



PDI: ATIVIDADE 02 · REGISTRO DE ENTREGA

# Resiliência **MarTech** no n8n Enterprise

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

n8n Enterprise / Supabase v3.0 / workflows-as-code / resiliência

## 1. Contexto

A operação MarTech da V4 Company processa ou requisições de integração com CRMs (Kommo, HubSpot, RD Station, entre outros) em workflows n8n. Três problemas estruturais motivam este PDI:

- **Workflows síncronos frágeis:** a requisição é processada dentro do webhook; em picos (campanha, importação), a instância trava e o cliente não recebe confirmação.
- **Payloads pesados sem checkpoint:** processamento de 10k+ itens de uma vez; um timeout no meio perde tudo e o job recomeça do zero.
- **Sincronização com CRM sem observabilidade:** falha não é registrada; divergência entre o esperado e o syncado só é descoberta quando o cliente reclama.

## 2. Diagnóstico

2.1 Causa raiz

Trabalho de integração tratado como **transação síncrona**. Sem fila, um pico vira travamento; sem checkpoint, um timeout vira retrabalho total; sem trilha de sync, a falha fica invisível até o impacto no cliente.

3. Solução em 3 frente

| FRENTE                 | SOLUÇÃO                                                                                                 | WORKFLOW                                       |
|------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------|
| 1. Fila + concorrência | Webhook enfileira e responde ACK 202 imediato; worker consome respeitando o semáforo de slots por fila. | [CC] MT Queue Gateway + [CC] MT - Queue Worker |
| 2. Payload pesado      | Processamento em chunks (JS) + enriquecimento (Python) com checkpoint retomável.                        | [CC] MT - Heavy Payload Processor              |
| 3. Observabilidade CRM | Auditoria de sync, health por objeto e detecção de divergência (drift) antes do cliente.                | [CC] MT - CRM Sync Observabilidade             |

```
// Fila assíncrona: respostasinho
POST /webhook/mt/gateway
→ 202 { "accepted": true, "status": "queued", "jobKey": "..." } // < 2s
```

4. Pipeline de processamento

| ETAPA         | RESPONSÁVEL                   | PERSISTÊNCIA                  |
|---------------|-------------------------------|-------------------------------|
| 1. Enfileirar | MT Queue Gateway              | mt_jobs (queued)              |
| 2. Consumir   | MT - Queue Worker (15s)       | slot mt_concurrency → running |
| 3. Processar  | MT - Heavy Payload Processor  | checkpoint mt_job_progress    |
| 4. Observar   | MT - CRM Sync Observabilidade | mt_sync_log + health + delta  |

## Como a engenharia funciona: o fluxo de ponta a ponta

● FLUXO EM EXECUÇÃO

1

**Cliente · CRM**

Dispara a `POST /webhook/mt/gateway`. No pico (campanha, importação), era aqui que a instância travava.

Kommo

HubSpot

RD Station

2

**MT Queue Gateway**

Não processa nada no webhook. Só enfileira e responde `ACK 202` imediato: o cliente não espera o pico passar.

`ACK 202 < 2s``idempotência job_key`

3

**Fila `mt_jobs` (Supabase)**

O job fica seguro no banco com estado, backoff e heartbeat. A instância pode reiniciar que nada se perde.

`queued → running``backoff 30s → 1m → 2m`

4

**MT - Queue Worker**

Polling a cada 15s. Consome slots respeitando o semáforo `mt_concurrency`: nunca estoura o limite da fila.

`schedule 15s``semáforo de slots`

5

**MT - Heavy Payload Processor**

Processa em chunks de `JS` + enriquecimento `Python` com checkpoint retomável. Falhou no chunk 4/10? Retoma do 4.

`chunking``checkpoint retomável`

6

**MT - CRM Sync Observabilidade**

Audita cada sync ( `mt_sync_log` ), mede health por objeto e calcula drift `expected × confirmed`. A divergência aparece **antes** do cliente reclamar.

`drift 14% > 5% → alerta``< 15 min`

Ciclo de vida de um job: o pulso percorre a linha enquanto a engrenagem processa.

```
● queued ● running ● done ● failed → retry
```

**Retomada:** falha no chunk 4/10 → o worker re-enfileira com backoff exponencial (30s → 1m → 2m, máx. 3) e o processamento retoma do chunk 4, não do zero.

## 5. Detecção de divergência (drift)

Todo sync envia um envelope padrão ( `expected` × `confirmed` ). O Observabilidade calcula o **drift %** e decide:

```
// Tolerância configurável: default 5%
expected    = 1000
confirmed   = 860
drift       = 14%

// 14% > 5% → mt_sync_delta (open) + alerta via dashboard
// Antes do impacto chegar ao cliente
```

## 6. Schema Supabase v3.0

| TABELA                       | USO                                                              |
|------------------------------|------------------------------------------------------------------|
| <code>mt_jobs</code>         | Fila assíncrona (queued/running/done/failed, backoff, heartbeat) |
| <code>mt_concurrency</code>  | Semáforo distribuído: limite por fila                            |
| <code>mt_job_progress</code> | Checkpoint de chunk de payload pesado                            |
| <code>mt_sync_log</code>     | Auditoria de cada sync de CRM                                    |
| <code>mt_crm_health</code>   | Saúde agregada por object + direction                            |
| <code>mt_sync_delta</code>   | Divergências abertas (drift)                                     |

**Aditivo:** não altera tabelas/views v2.x ( `error_*` ) da atividade 1. Convive lado a lado.

**Views:** `vw_mt_queue_backlog` · `vw_mt_slots` · `vw_mt_sync_summary_24h` · `vw_mt_drift_abertos`  
· `vw_mt_crm_health`

## 7. Workflows entregues

| WORKFLOW                           | TIPO                 | FUNÇÃO                                         |
|------------------------------------|----------------------|------------------------------------------------|
| [CC] MT Queue Gateway              | Webhook              | Enfileira + ACK 202 + idempotência por job_key |
| [CC] MT - Queue Worker             | Schedule 15s         | Consome slots, delega, faz retry com backoff   |
| [CC] MT - Heavy Payload Processor  | Sub-workflow         | Chunks JS + Python, checkpoint e retomada      |
| [CC] MT - CRM Sync Observabilidade | Webhook /mt/crm-sync | Log, health e drift                            |

## 8. Métricas de sucesso

| MÉTRICA                     | ANTES                    | META                    |
|-----------------------------|--------------------------|-------------------------|
| ACK de requisição MarTech   | Trava no webhook         | < 2s                    |
| Job pesado retomável        | Não                      | Sim                     |
| Falha de sync detectada     | Quando o cliente reclama | < 15 min                |
| Divergência (drift) visível | Não                      | Dashboard em tempo real |
| Concorrência controlada     | Manual/improvisada       | Semáforo mt_concurrency |

## 9. Entregas

| PASTA        | CONTEÚDO                                                             | FORMATO      |
|--------------|----------------------------------------------------------------------|--------------|
| 1-standards/ | 3 padrões: filas/concorrência, payloads pesados, observabilidade CRM | Markdown     |
| 2-workflows/ | 4 workflows validados com n8nac                                      | .workflow.ts |

|                                 |                                                      |                 |
|---------------------------------|------------------------------------------------------|-----------------|
| <a href="#">3-supabase/</a>     | Schema v3.0 (6 tabelas + 5 views) + guia de migração | SQL + Markdown  |
| <a href="#">4-retrofit/</a>     | Como adaptar workflows MarTech existentes            | Markdown        |
| <a href="#">5-monitoring/</a>   | Consultas SQL de dashboard + alertas                 | Markdown        |
| <a href="#">6-automation/</a>   | Scripts de migração e deploy (com gate de aprovação) | Shell           |
| <a href="#">7-apresentacao/</a> | Deck HTML + demo + este relatório                    | HTML + Markdown |

## 10. Decisoes e tradeoffs

Escolhas derivadas do conteudo real da atividade, com custo e beneficio explicitos.

- **Gateway que so enfileira com ACK 202 em menos de 2s:** absorve pico sem travar a instancia. O cliente recebe confirmacao de fila (queued) e consulta mt\_jobs para o status final.
- **Worker com polling a cada 15s e semaforo em mt\_concurrency:** limita pressao sobre n8n e CRMs. Latencia minima de ciclo em troca de nunca estourar o limite da fila.
- **Checkpoint retomavel com backoff de 30s, 1m e 2m e maximo de 3 tentativas:** falha no chunk 4 de 10 retoma do chunk 4. Mais escritas no Supabase por chunk em troca de zero retrabalho total.
- **Schema v3.0 aditivo com 6 tabelas e 5 views:** convive com as tabelas error da atividade 1 sem migracao destrutiva. Mais tabelas para operar em troca de deploy seguro.
- **Drift com tolerancia default de 5%:** exemplo com 1000 esperados e 860 confirmados gera 14% e abre mt\_sync\_delta antes do cliente reclamar. Limiar fixo pode alertar em variacao legitima, em troca de deteccao em menos de 15 min.

## 11. Impacto no negocio

Antes, pico de campanha ou importacao travava a instancia no webhook, payload de 10k itens sem checkpoint perdia tudo no timeout e falha de sync com Kommo, HubSpot ou RD Station so aparecia na reclamacao. Com fila, ACK em menos de 2s, processamento retomavel e drift visivel em dashboard, a meta sai de volume nao suportado para 5x o volume nominal, de travamento por concorrencia ilimitada para limite por slot e de deteccao na reclamacao para menos de 15 min, com mais de 90% dos payloads pesados processados. Isso reduz risco de indisponibilidade em pico e custo de retrabalho de jobs refeitos do zero.

## 12. Referencias de estudo

Material para aprofundar os temas usados nesta atividade.

- **Curso:** Fundamentos de Filas e Sistemas Assíncronos, na Alura.
- **Video:** n8n Webhooks and Queue Pattern Explained, no YouTube, canal oficial n8n.
- **Doc:** n8n Docs, Webhook node and sub-workflows, na plataforma n8n Docs.
- **Doc:** Supabase Docs, Postgres tables and views, na plataforma Supabase Docs.

**Status:** desenvolvido · workflow e schema prontos para produção · aguardando homologação. Nenhum workflow foi publicado no n8n nesta etapa.